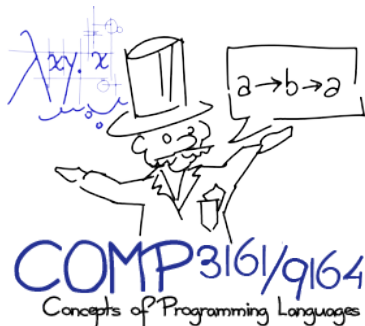




Existential Types and Abstraction

Johannes Åman Pohjola
UNSW
Term 3 2023

Motivation

Throughout your studies, lecturers have (hopefully) expounded on the software engineering advantages of *abstract data types*.

So what is an *abstract data type*?

Motivation

Throughout your studies, lecturers have (hopefully) expounded on the software engineering advantages of *abstract data types*.

So what is an *abstract data type*?

Definition

An *abstract data type* is a type defined not by its internal representation, but by the operations that can be performed on it.

Typically, these operations are specified using a more abstract model than the actual implementation.

Language Examples: C

How do we do it in C?

Language Examples: C

How do we do it in C?

stack.h

```
typedef stack_impl *Stack;
```

```
Stack empty();
```

```
Stack push(Stack, int);
```

```
Stack pop(Stack, int*);
```

```
bool isEmpty(Stack);
```

```
void destroy(Stack);
```

Language Examples: C

How do we do it in C?

stack.h

```
typedef stack_impl *Stack;
```

```
Stack empty();
```

```
Stack push(Stack, int);
```

```
Stack pop(Stack, int*);
```

```
bool isEmpty(Stack);
```

```
void destroy(Stack);
```

stack.c

```
#include "stack.h"
```

```
struct stack_impl {  
    int head;  
    Stack tail;  
}
```

```
Stack empty() { ... }
```

```
...
```

Language Examples: C

How do we do it in C?

stack.h

```
typedef stack_impl *Stack;
```

```
Stack empty();  
Stack push(Stack, int);  
Stack pop(Stack, int*);  
bool isEmpty(Stack);  
void destroy(Stack);
```

By only importing stack.h,
we hide the implementation.

stack.c

```
#include "stack.h"  
  
struct stack_impl {  
    int head;  
    Stack tail;  
}  
  
Stack empty() { ... }  
...
```

Language Examples: Haskell

Define a module but restrict what is exported:

```
module Stack
  ( Stack -- Cons and Nil are *not* exported
  , empty
  , push
  , pop
  , isEmpty
  ) where

data Stack = Cons Int Stack | Nil

empty :: Stack
empty = Nil

...
```


Language Examples: Java

Typically Java accomplishes this with **subtype polymorphism**, something we discuss in the next lecture.

```
public interface Stack {  
    public void push(int x);  
    public int pop() throws EmptyStackException;  
    public boolean isEmpty();  
}
```

```
public class ListStack implements Stack {  
    public ListStack() { ... };  
  
    ...  
}
```

Language Examples: Python

Language Examples: Python

No luck here.

Quote

“Python is very simple and nice when you start to use it, but you don’t get too far down the road, if you’re me, before you discover it has no data abstraction at all. That’s not good because big programs require modularity and encapsulation and you’d like a language that could support that.”

Barbara Liskov, *The Power of Abstraction*, 2013.

You don’t need static types to enforce abstraction, but it helps.

MinHS

How can we support abstract data types in MinHS? Can we use existing features to do so?

MinHS

How can we support abstract data types in MinHS? Can we use existing features to do so? **We can use parametric polymorphism:**

```
(type S.  
  recfun foo push pop isEmpty empty =  
    let s = push empty 42  
    in isEmpty (fst (pop s)))  
  ::
```

MinHS

How can we support abstract data types in MinHS? Can we use existing features to do so? **We can use parametric polymorphism:**

(**type** $\mathcal{S}$.

recfun *foo push pop isEmpty empty =*

let $s = \text{push } \text{empty } 42$

in $\text{isEmpty } (\text{fst } (\text{pop } s))$)

$::$

$\forall \mathcal{S}. (\mathcal{S} \rightarrow \text{Int} \rightarrow \mathcal{S})$ (push)

$\rightarrow (\mathcal{S} \rightarrow \mathcal{S} \times \text{Int})$ (pop)

$\rightarrow (\mathcal{S} \rightarrow \text{Bool})$ (isEmpty)

$\rightarrow \mathcal{S}$ (empty)

$\rightarrow \text{Bool}$

MinHS

How can we support abstract data types in MinHS? Can we use existing features to do so? **We can use parametric polymorphism:**

```
(type  $\mathcal{S}$ .
  recfun foo push pop isEmpty empty =
    let  $s = \text{push empty } 42$ 
    in  $\text{isEmpty (fst (pop } s))$ )
  ::
 $\forall \mathcal{S}. (\mathcal{S} \rightarrow \text{Int} \rightarrow \mathcal{S})$            (push)
 $\rightarrow (\mathcal{S} \rightarrow \mathcal{S} \times \text{Int})$              (pop)
 $\rightarrow (\mathcal{S} \rightarrow \text{Bool})$                  (isEmpty)
 $\rightarrow \mathcal{S}$                              (empty)
 $\rightarrow \text{Bool}$ 
```

The program *foo* is defined for any stack type $\mathcal{S}$. Implementations of the operations must be provided as **parameters**.

Modules

We would like a single value to pass around, that contains the whole stack interface. It's too cumbersome to pass around each component individually like before. This value is called a *module*.

Modules

We would like a single value to pass around, that contains the whole stack interface. It's too cumbersome to pass around each component individually like before. This value is called a *module*.

Our toy *foo* program from earlier needs to be rewritten as:

$$\text{STACKMODULE} \rightarrow \text{Bool}$$

For some type `STACKMODULE`. Taking in a value of type `STACKMODULE` is analogous to *importing* the module.

Via Curry-Howard

Let's translate the type of *foo* into a proposition, then do logical transformations to it: Perhaps do this on the whiteboard.

$$\forall S. ((S \rightarrow \text{Int} \rightarrow S) \rightarrow (S \rightarrow S \times \text{Int}) \rightarrow (S \rightarrow \text{Bool}) \rightarrow S \rightarrow \text{Bool})$$

Via Curry-Howard

Let's translate the type of *foo* into a proposition, then do logical transformations to it: Perhaps do this on the whiteboard.

$$\forall \mathcal{S}. ((\mathcal{S} \rightarrow \text{Int} \rightarrow \mathcal{S}) \rightarrow (\mathcal{S} \rightarrow \mathcal{S} \times \text{Int}) \rightarrow (\mathcal{S} \rightarrow \text{Bool}) \rightarrow \mathcal{S} \rightarrow \text{Bool})$$

(translating to logic)

Via Curry-Howard

Let's translate the type of *foo* into a proposition, then do logical transformations to it: Perhaps do this on the whiteboard.

$$\forall S. ((S \rightarrow \text{Int} \rightarrow S) \rightarrow (S \rightarrow S \times \text{Int}) \rightarrow (S \rightarrow \text{Bool}) \rightarrow S \rightarrow \text{Bool})$$

(translating to logic)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \Rightarrow (S \Rightarrow S \wedge \text{Int}) \Rightarrow (S \Rightarrow \text{Bool}) \Rightarrow S \Rightarrow \text{Bool})$$

Via Curry-Howard

Let's translate the type of *foo* into a proposition, then do logical transformations to it: Perhaps do this on the whiteboard.

$$\forall S. ((S \rightarrow \text{Int} \rightarrow S) \rightarrow (S \rightarrow S \times \text{Int}) \rightarrow (S \rightarrow \text{Bool}) \rightarrow S \rightarrow \text{Bool})$$

(translating to logic)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \Rightarrow (S \Rightarrow S \wedge \text{Int}) \Rightarrow (S \Rightarrow \text{Bool}) \Rightarrow S \Rightarrow \text{Bool})$$

(as $P \Rightarrow Q \Rightarrow R = P \wedge Q \Rightarrow R$)

Via Curry-Howard

Let's translate the type of *foo* into a proposition, then do logical transformations to it: Perhaps do this on the whiteboard.

$$\forall S. ((S \rightarrow \text{Int} \rightarrow S) \rightarrow (S \rightarrow S \times \text{Int}) \rightarrow (S \rightarrow \text{Bool}) \rightarrow S \rightarrow \text{Bool})$$

(translating to logic)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \Rightarrow (S \Rightarrow S \wedge \text{Int}) \Rightarrow (S \Rightarrow \text{Bool}) \Rightarrow S \Rightarrow \text{Bool})$$

(as $P \Rightarrow Q \Rightarrow R = P \wedge Q \Rightarrow R$)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \wedge (S \Rightarrow S \wedge \text{Int}) \wedge (S \Rightarrow \text{Bool}) \wedge S \Rightarrow \text{Bool})$$

Via Curry-Howard

Let's translate the type of *foo* into a proposition, then do logical transformations to it: Perhaps do this on the whiteboard.

$$\forall S. ((S \rightarrow \text{Int} \rightarrow S) \rightarrow (S \rightarrow S \times \text{Int}) \rightarrow (S \rightarrow \text{Bool}) \rightarrow S \rightarrow \text{Bool})$$

(translating to logic)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \Rightarrow (S \Rightarrow S \wedge \text{Int}) \Rightarrow (S \Rightarrow \text{Bool}) \Rightarrow S \Rightarrow \text{Bool})$$

(as $P \Rightarrow Q \Rightarrow R = P \wedge Q \Rightarrow R$)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \wedge (S \Rightarrow S \wedge \text{Int}) \wedge (S \Rightarrow \text{Bool}) \wedge S \Rightarrow \text{Bool})$$

(as $\forall X.(P(X) \Rightarrow Q) = (\exists X.P(X)) \Rightarrow Q$)

Via Curry-Howard

Let's translate the type of *foo* into a proposition, then do logical transformations to it: Perhaps do this on the whiteboard.

$$\forall S. ((S \rightarrow \text{Int} \rightarrow S) \rightarrow (S \rightarrow S \times \text{Int}) \rightarrow (S \rightarrow \text{Bool}) \rightarrow S \rightarrow \text{Bool})$$

(translating to logic)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \Rightarrow (S \Rightarrow S \wedge \text{Int}) \Rightarrow (S \Rightarrow \text{Bool}) \Rightarrow S \Rightarrow \text{Bool})$$

(as $P \Rightarrow Q \Rightarrow R = P \wedge Q \Rightarrow R$)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \wedge (S \Rightarrow S \wedge \text{Int}) \wedge (S \Rightarrow \text{Bool}) \wedge S \Rightarrow \text{Bool})$$

(as $\forall X. (P(X) \Rightarrow Q) = (\exists X. P(X)) \Rightarrow Q$)

$$(\exists S. (S \Rightarrow \text{Int} \Rightarrow S) \wedge (S \Rightarrow S \wedge \text{Int}) \wedge (S \Rightarrow \text{Bool}) \wedge S) \Rightarrow \text{Bool}$$

Via Curry-Howard

Let's translate the type of *foo* into a proposition, then do logical transformations to it: Perhaps do this on the whiteboard.

$$\forall S. ((S \rightarrow \text{Int} \rightarrow S) \rightarrow (S \rightarrow S \times \text{Int}) \rightarrow (S \rightarrow \text{Bool}) \rightarrow S \rightarrow \text{Bool})$$

(translating to logic)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \Rightarrow (S \Rightarrow S \wedge \text{Int}) \Rightarrow (S \Rightarrow \text{Bool}) \Rightarrow S \Rightarrow \text{Bool})$$

(as $P \Rightarrow Q \Rightarrow R = P \wedge Q \Rightarrow R$)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \wedge (S \Rightarrow S \wedge \text{Int}) \wedge (S \Rightarrow \text{Bool}) \wedge S \Rightarrow \text{Bool})$$

(as $\forall X. (P(X) \Rightarrow Q) = (\exists X. P(X)) \Rightarrow Q$)

$$(\exists S. (S \Rightarrow \text{Int} \Rightarrow S) \wedge (S \Rightarrow S \wedge \text{Int}) \wedge (S \Rightarrow \text{Bool}) \wedge S) \Rightarrow \text{Bool}$$

(back to types)

Via Curry-Howard

Let's translate the type of *foo* into a proposition, then do logical transformations to it: Perhaps do this on the whiteboard.

$$\forall S. ((S \rightarrow \text{Int} \rightarrow S) \rightarrow (S \rightarrow S \times \text{Int}) \rightarrow (S \rightarrow \text{Bool}) \rightarrow S \rightarrow \text{Bool})$$

(translating to logic)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \Rightarrow (S \Rightarrow S \wedge \text{Int}) \Rightarrow (S \Rightarrow \text{Bool}) \Rightarrow S \Rightarrow \text{Bool})$$

(as $P \Rightarrow Q \Rightarrow R = P \wedge Q \Rightarrow R$)

$$\forall S. ((S \Rightarrow \text{Int} \Rightarrow S) \wedge (S \Rightarrow S \wedge \text{Int}) \wedge (S \Rightarrow \text{Bool}) \wedge S \Rightarrow \text{Bool})$$

(as $\forall X. (P(X) \Rightarrow Q) = (\exists X. P(X)) \Rightarrow Q$)

$$(\exists S. (S \Rightarrow \text{Int} \Rightarrow S) \wedge (S \Rightarrow S \wedge \text{Int}) \wedge (S \Rightarrow \text{Bool}) \wedge S) \Rightarrow \text{Bool}$$

(back to types)

$$(\exists S. (S \rightarrow \text{Int} \rightarrow S) \times (S \rightarrow S \times \text{Int}) \times (S \rightarrow \text{Bool}) \times S) \rightarrow \text{Bool}$$

Existential Types

We have our STACKMODULE type:

$$(\exists \mathcal{S}. (\mathcal{S} \rightarrow \text{Int} \rightarrow \mathcal{S}) \times (\mathcal{S} \rightarrow \mathcal{S} \times \text{Int}) \times (\mathcal{S} \rightarrow \text{Bool}) \times \mathcal{S}) \rightarrow \text{Bool}$$

STACKMODULE

But what is this $\exists a. \tau$ thing?

Existential Types

We have our `STACKMODULE` type:

$$(\exists \mathcal{S}. (\mathcal{S} \rightarrow \text{Int} \rightarrow \mathcal{S}) \times (\mathcal{S} \rightarrow \mathcal{S} \times \text{Int}) \times (\mathcal{S} \rightarrow \text{Bool}) \times \mathcal{S}) \rightarrow \text{Bool}$$

`STACKMODULE`

But what is this $\exists a. \tau$ thing?

Existential vs Universal Types

$\forall a. \tau$ When **producing** a value, a is an arbitrary, unknown type.
When **consuming** a value, a may be instantiated to any desired type.

$\exists a. \tau$ When **consuming** a value, a is an arbitrary, unknown type.
When **producing** a value, a may be instantiated to any desired type.

Another, Smaller Example

An ADT Bag is specified by three operations:

- ① *emptyBag*, which gives a new, empty bag.
- ② *addToBag*, which adds an integer to the bag.
- ③ *average*, which gives the arithmetic mean of the bag.

What's the type for this?

Another, Smaller Example

An ADT Bag is specified by three operations:

- ① *emptyBag*, which gives a new, empty bag.
- ② *addToBag*, which adds an integer to the bag.
- ③ *average*, which gives the arithmetic mean of the bag.

What's the type for this?

$$\text{BAGMODULE} = \exists \mathcal{B}. \underbrace{\mathcal{B}}_{\text{emptyBag}} \times \overbrace{(\mathcal{B} \rightarrow \text{Int} \rightarrow \mathcal{B})}^{\text{addToBag}} \times \overbrace{(\mathcal{B} \rightarrow \text{Int})}^{\text{average}}$$

The type of a module is called its *signature*.

Making a Module

We can make a value of an existential type using the **Pack** expression.

$$\frac{\Delta \vdash \tau \text{ ok} \quad \Delta; \Gamma \vdash e : \rho[a := \tau]}{\Delta; \Gamma \vdash (\text{Pack } \tau \ e) : \exists a. \rho}$$

Just as the type $\forall a. \tau$ could be viewed as a **function** from a type to a value, the type $\exists a. \tau$ could be viewed as a **pair** of a type and a value.

Making a Module

We can make a value of an existential type using the `Pack` expression.

$$\frac{\Delta \vdash \tau \text{ ok} \quad \Delta; \Gamma \vdash e : \rho[a := \tau]}{\Delta; \Gamma \vdash (\text{Pack } \tau \ e) : \exists a. \rho}$$

Just as the type $\forall a. \tau$ could be viewed as a **function** from a type to a value, the type $\exists a. \tau$ could be viewed as a **pair** of a type and a value.

Example (Bag as two integers)

```
Pack (Int × Int)
( (0, 0)
, recfun addToBag b i = (fst b + i, snd b + 1)
, recfun average b = (fst b ÷ snd b)
) :: BAGMODULE
```


Importing a Module

If we have a module, we can access its contents using `Open`:

$$\frac{\begin{array}{c} \Delta; \Gamma \vdash e_1 : \exists a. \tau \quad (\Delta, \textcolor{red}{a} \text{ bound}); (\Gamma, x : \textcolor{red}{\tau}) \vdash e_2 : \rho \\ (a \text{ bound}) \notin \Delta \quad \Delta \vdash \rho \text{ ok} \end{array}}{\Delta; \Gamma \vdash (\text{Open } e_1 \ (a. x. e_2)) : \rho}$$

The last two premises ensure that the type ρ does not contain the abstract type—it is only in scope inside e_2 .

Importing a Module

If we have a module, we can access its contents using `Open`:

$$\frac{\Delta; \Gamma \vdash e_1 : \exists a. \tau \quad (\Delta, \textcolor{red}{a} \text{ bound}); (\Gamma, x : \textcolor{red}{\tau}) \vdash e_2 : \rho \quad (a \text{ bound}) \notin \Delta \quad \Delta \vdash \rho \text{ ok}}{\Delta; \Gamma \vdash (\text{Open } e_1 \ (a. x. e_2)) : \rho}$$

The last two premises ensure that the type ρ does not contain the abstract type—it is only in scope inside e_2 .

Example (Averaging some numbers with a bag)

```

recfun f :: (BAGMODULE → Int) bagM =
  Open bagM
    (B. (empty, addToBag, average).
      average (addToBag (addToBag empty 60) 30)
    )
  
```

Type inference?

Full type inference for existential types is an open research problem.

```
recfun  $f$   $b =$   
  if  $b$  then  
     $(1, \lambda y. y + 1)$   
  else  
     $(\text{true}, \lambda y. 1)$ 
```

Q: What's the type of f ?

Type inference?

Full type inference for existential types is an open research problem.

```
recfun  $f$   $b =$   
  if  $b$  then  
     $(1, \lambda y. y + 1)$   
  else  
     $(\text{true}, \lambda y. 1)$ 
```

Q: What's the type of f ?

A: Either of these:

$\text{Bool} \rightarrow \exists a. a \times (a \rightarrow \text{Int})$

$\text{Bool} \rightarrow \exists a. a \times (\text{Int} \rightarrow \text{Int})$

...but neither is more general.

Type inference?

Full type inference for existential types is an open research problem.

```
recfun  $f$   $b =$   
  if  $b$  then  
     $(1, \lambda y. y + 1)$   
  else  
     $(\text{true}, \lambda y. 1)$ 
```

Q: What's the type of f ?

A: Either of these:

$\text{Bool} \rightarrow \exists a. a \times (a \rightarrow \text{Int})$

$\text{Bool} \rightarrow \exists a. a \times (\text{Int} \rightarrow \text{Int})$

...but neither is more general.

Algorithms do exist with additional restrictions or annotations. See e.g. Eisenberg et. al, ICFP 2021.

In Practice

Programming language support for modules is a mixed bag.

- Dynamically typed languages typically don't support them at all¹.
- Haskell without extensions, C, and Go have very weak support for them.
- Rust has a feature called *impl Traits* which are a limited form of existential types.
- Java and similar accomplish modularity via OOP, which don't support existential typing in its full generality.
- Languages in the ML family, like SML and OCaml have very good support for modules, but typically not modules-as-expressions.

¹What they call “modules” aren't. Just like types.